



Universidade Tecnológica Federal do Paraná
Câmpus Pato Branco

ESPECIALIZAÇÃO EM DESENVOLVIMENTO PARA DISPOSITIVOS MÓVEIS

MATHAUS LAZARUS
MÔNICA MEDEIROS
NARDÉLIO FERREIRA DOS SANTOS

***AWS AMPLIFY PARA ANDROID: CONECTIVIDADE COM
SERVIÇOS DA AMAZON
AUTH, STORAGE E API GATEWAY EM UM ESTUDO DE CASO***

PATO BRANCO
2026

1. INTRODUÇÃO

O desenvolvimento mobile vai além de apenas construir telas. Atualmente, um aplicativo funcional precisa realizar diversas outras ações, como autenticar usuários, armazenar arquivos, consumir *APIs* e até se comunicar com serviços na nuvem. Em um desenvolvimento tradicional, isso significa criar servidores, configurar banco de dados, implementar autenticações e manter toda essa infraestrutura em produção. Para equipes menores, esse conjunto de obrigações pode se tornar maior que o próprio desenvolvimento do app.

Foi nesse cenário que nasceram as soluções *Backend as a Service (BaaS)* e as arquiteturas *serverless*. A ideia é consumir serviços prontos via *SDKs* e *APIs* ao invés de gerenciar servidores, mantendo a responsabilidade no provedor. No desenvolvimento mobile, isso permite que os desenvolvedores possam focar na experiência do usuário e nas regras de negócio.

Este artigo explora como o *AWS Amplify* se encaixa nesse contexto para aplicações *Android*. O foco deste trabalho está nos três recursos que mais aparecem em projetos reais: a autenticação através do *Amplify Auth* com *Amazon Cognito*, o armazenamento de arquivos via *Amplify Storage* e a comunicação com o *Backend* via *Amplify API* com *Amazon API Gateway*.

2. VISÃO GERAL DO AWS AMPLIFY

2.1 Conceito

O *AWS Amplify* é um conjunto de ferramentas e bibliotecas da *Amazon* voltado para conectar aplicações mobile e web a serviços da *AWS*. No *Android*, ele funciona como uma camada entre o aplicativo e recursos como autenticação, armazenamento e *APIs* - permitindo usar esses serviços sem precisar configurar manualmente do zero.

2.2 Configuração básica em um projeto *Android*

Para utilizar o *AWS Amplify* em uma aplicação *Android*, é necessário realizar uma configuração inicial. De forma geral, primeiro são definidos os recursos de *Backend* que a aplicação utilizará, como autenticação, armazenamento de arquivos ou *API*. Esses recursos são configurados na *AWS* e depois conectados ao projeto *Android*.

Essa conexão é feita por meio do arquivo `amplify_outputs.json`, que é gerado após a configuração do *Backend* e deve ser adicionado ao diretório `app/src/main/res/raw` do projeto *Android*. Esse arquivo contém dados do *Backend*, como *endpoints*, informações de autenticação e configurações usadas pela biblioteca cliente do *Amplify* para se conectar aos recursos criados.

Além do arquivo de configuração, o projeto *Android* precisa incluir as bibliotecas do *Amplify* e inicializar os *plugins* correspondentes às funcionalidades utilizadas. Por exemplo, uma aplicação que utiliza autenticação precisa configurar o *plugin* relacionado ao *Auth*; uma aplicação que utiliza armazenamento precisa configurar o *plugin* relacionado ao *Storage*; e uma aplicação que consome *APIs* precisa configurar o *plugin* relacionado à categoria *API*.

O exemplo a seguir apresenta, de forma simplificada, a inicialização do *Amplify* em uma aplicação *Android* utilizando o *plugin* de autenticação:

```
// Exemplo adicionando o plugin de autenticação
class MyAmplifyApp : Application() {
    override fun onCreate() {
        super.onCreate()
        try {
            Amplify.addPlugin(AWSCognitoAuthPlugin())
            Amplify.configure(
                AmplifyOutputs(R.raw.amplify_outputs),
                applicationContext
            )
            Log.i("MyAmplifyApp", "Amplify inicializado")
        } catch (error: AmplifyException) {
            Log.e("MyAmplifyApp", "Erro ao inicializar o Amplify", error)
        }
    }
}
```

Depois dessa configuração inicial, o código do aplicativo pode utilizar chamadas como *Amplify.Auth* para acessar o serviço de autenticação configurado na *AWS*.

Do ponto de vista técnico, o *AWS Amplify* atua como uma camada de abstração sobre os *SDKs* da *AWS*, simplificando a comunicação entre o aplicativo e os serviços de *Backend*. Internamente, essa comunicação ocorre por meio de chamadas autenticadas utilizando credenciais temporárias ou *tokens* de acesso, dependendo do serviço utilizado.

No caso da autenticação, o *Amazon Cognito* utiliza *tokens* do tipo JWT (*JSON Web Token*), que são emitidos após o processo de login e utilizados para autenticar requisições subsequentes. Esses *tokens* carregam informações sobre o usuário e suas permissões, permitindo que o *Backend* valide o acesso sem necessidade de manter estado de sessão.

Além disso, o uso do *Amplify* reduz a necessidade de implementação manual de chamadas *REST* ou configuração de *SDKs* específicos para cada serviço, centralizando a integração em uma *API* unificada. No entanto, essa abstração implica em menor visibilidade sobre detalhes de baixo nível, o que pode impactar cenários que exigem controle mais granular da infraestrutura.

3. OS TRÊS PILARES DO *AMPLIFY* NO *ANDROID*

3.1 *Amplify Auth* e *Amazon Cognito*

A autenticação é um dos recursos mais importantes em aplicativos. No *AWS Amplify*, o *Auth* permite integrar o aplicativo ao *Amazon Cognito* - um serviço que lida com a autenticação e autorização de usuários.

Em uma aplicação *Android*, o *Amplify Auth* permite implementar diversas operações, como cadastro de usuários, *login*, *logout*, recuperação de conta e verificação da sessão autenticada, utilizando a camada de integração com o serviço de autenticação da *AWS*.

No fluxo de cadastro, por exemplo, o aplicativo recebe informações como e-mail, telefone, nome de usuário e senha, e envia esses dados para o *Amazon Cognito* por meio da biblioteca do *Amplify*. Após o cadastro, dependendo da configuração adotada, pode ser necessário confirmar a conta utilizando um código enviado por e-mail ou SMS.

O exemplo a seguir apresenta, de forma simplificada, o cadastro de um novo usuário utilizando o *Amplify Auth* em uma aplicação *Android*:

```
// Cadastro de novo usuário
val attrs = mapOf(
    AuthUserAttributeKey.email() to "usuario@email.com",
    AuthUserAttributeKey.phoneNumber() to "+5511999999999"
)
val options = AuthSignUpOptions.builder()
    .userAttributes(attrs.map { AuthUserAttribute(it.key, it.value) })
    .build()
```

```
try {
    val result = Amplify.Auth.signUp(
        "usuario",
        "Senha123",
        options
    )
    Log.i("Auth", "Cadastro iniciado: $result")
} catch (error: AuthException) {
    Log.e("Auth", "Falha ao cadastrar usuário", error)
}
```

Fonte: Adaptado da documentação oficial do *AWS Amplify para Android*

Nesse exemplo, os atributos do usuário são definidos e enviados ao *Cognito* durante a chamada de cadastro pelo *Amplify Auth*. Caso seja concluído com sucesso, o aplicativo pode prosseguir para a etapa de confirmação da conta, conforme a configuração do serviço.

Em termos de segurança, o *Amazon Cognito* utiliza *tokens* JWT para autenticação e autorização. Esses *tokens* são enviados nas requisições realizadas pelo aplicativo e podem ser validados pelo *Backend*, garantindo que apenas usuários autenticados acessem determinados recursos. Além disso, o *Cognito* permite a definição de políticas de acesso e integração com provedores externos, aumentando a flexibilidade e segurança do sistema.

3.2 *Amplify Storage* e *AmazonS3*

O armazenamento de arquivos é necessário em diversos tipos de aplicativos, como envio de fotos de perfil, documentos, comprovantes ou imagens de produtos.

Em uma aplicação *Android*, o *Amplify Storage* permite realizar operações como envio, download, listagem e remoção de arquivos. Dessa forma, o aplicativo pode trabalhar com arquivos locais e realizar o envio para a nuvem por meio da biblioteca fornecida pelo *Amplify*, sem que seja necessário implementar uma comunicação direta com o *Amazon S3*.

Quando se trata de *upload*, o aplicativo pode selecionar ou gerar um arquivo local, abrir um fluxo de leitura e enviar esse conteúdo para o armazenamento configurado na *AWS*. Dependendo da configuração do projeto, os arquivos podem ter diferentes regras de acesso, como públicos, protegidos ou privados.

O exemplo a seguir apresenta uma função que tem como objetivo realizar o envio de um arquivo local utilizando o *Amplify Storage* em uma aplicação *Android*:

// Função para realizar o upload de arquivos para o S3 utilizando amplify

```
private suspend fun uploadFile() {  
    val exampleFile = File(applicationContext.filesDir, "exemploUtfpr")  
    exampleFile.writeText("Exemplo de arquivo com conteúdo")  
    val upload = Amplify.Storage.uploadFile(  
        StoragePath.fromString("public/exemplo"),  
        exampleFile  
    )  
    try {  
        val result = upload.result()  
        Log.i("Storage", "Arquivo enviado: ${result.path}")  
    } catch (error: StorageException) {  
        Log.e("Storage", "Falha no upload", error)  
    }  
}
```

Fonte: Adaptado da documentação oficial do AWS Amplify para Android.

No exemplo, o método *uploadFile()* envia um arquivo local para o armazenamento configurado no *Amplify*. Esse armazenamento é associado ao *Amazon S3*, serviço de armazenamento de objetos da *AWS*.

A segurança no armazenamento é gerenciada por meio de políticas de acesso associadas ao *Amazon S3* e integradas ao *Amazon Cognito*. Dessa forma, é possível definir diferentes níveis de acesso aos arquivos, como público, protegido ou privado. Esse modelo garante que usuários autenticados tenham acesso apenas aos seus próprios dados, reduzindo riscos de exposição indevida de informações.

Além disso, o controle de acesso aos arquivos é gerenciado por meio de *IAM Roles* e políticas (*policies*), permitindo definir permissões específicas para cada usuário autenticado. Esse mecanismo garante maior segurança no acesso aos objetos armazenados e reduz riscos de exposição indevida de dados.

3.3 Amplify API e Amazon API Gateway

Operações como criar pedidos, consultar dados, validar permissões ou executar integrações externas normalmente pertencem ao *Backend*. No *AWS Amplify*, essa funcionalidade pode ser utilizada através do *Amplify API*, que permite ao aplicativo realizar requisições a serviços de *Backend*.

Em uma aplicação *Android*, o *Amplify API* pode ser utilizado para realizar chamadas *HTTP*, como *GET*, *POST*, *PUT* e *DELETE*, para uma *API* configurada na *AWS*. No caso de uma *API REST* criada com *Amplify*, a comunicação pode envolver o *Amazon API Gateway*, responsável por expor as rotas da *API*, e o *AWS Lambda*, responsável por executar a lógica de negócio no *Backend*.

O *Amazon API Gateway* atua como uma camada de entrada para a *API*, permitindo que o aplicativo acesse funcionalidades do *Backend* por meio de rotas *HTTP*. Já o *AWS Lambda* permite executar código sem que a equipe precise gerenciar servidores diretamente. Dessa forma, o aplicativo *Android* não precisa acessar diretamente banco de dados, funções internas ou serviços externos; ele consome uma *API* que funciona como contrato entre o cliente e o *Backend*.

O exemplo a seguir mostra uma chamada *GET* utilizando o *Amplify API* em uma aplicação *Android*:

```
// Função para realizar uma requisição get
private suspend fun getPedidos() {
    val request = RestOptions.builder()
        .addPath("/pedidos")
        .build()
    try {
        val response = Amplify.API.get(request)
        Log.i("Resposta:", "GET sucesso: ${response.data}")
    } catch (error: ApiException) {
        Log.e("Erro:", "GET erro", error)
    }
}
```

Fonte: Adaptado da documentação oficial do *AWS Amplify para Android*

Nesse exemplo, o aplicativo cria uma requisição para a rota de */pedidos* e utiliza o método *get()* da *Amplify API* para enviar a chamada ao *Backend* configurado.

No contexto da *API*, a segurança é reforçada pelo uso de mecanismos de autorização no *Amazon API Gateway*, como o uso de *authorizers* baseados em *tokens* do *Cognito*. Isso permite validar cada requisição recebida antes de encaminhá-la para o *Backend*, garantindo que apenas chamadas autenticadas sejam processadas pelas funções *AWS Lambda*.

4. ESTUDO DE CASO

Para exemplificar o uso conjunto dos recursos apresentados, utilizamos o exemplo de um aplicativo de *delivery* de uma pizzaria. Nesse aplicativo, os usuários poderiam consultar o cardápio, realizar pedidos e acompanhar o status da compra.

Esse cenário foi escolhido por reunir funcionalidades comuns em aplicações *mobile* conectadas à nuvem.

A autenticação permite identificar o usuário antes da realização do pedido; o armazenamento possibilita o envio de arquivos, como foto de perfil ou comprovante; e a *API* concentra a comunicação entre o aplicativo e as regras de negócio do *Backend*.

Antes do uso no aplicativo *Android*, os recursos necessários seriam configurados na *AWS*. O projeto teria autenticação com *Amazon Cognito*, armazenamento de arquivos com *Amazon S3* e uma *API* para comunicação com as regras de negócio. Depois dessa configuração, o aplicativo *Android* poderia utilizar as bibliotecas do *Amplify* para acessar esses recursos.

A arquitetura da solução proposta pode ser observada na Figura 1.

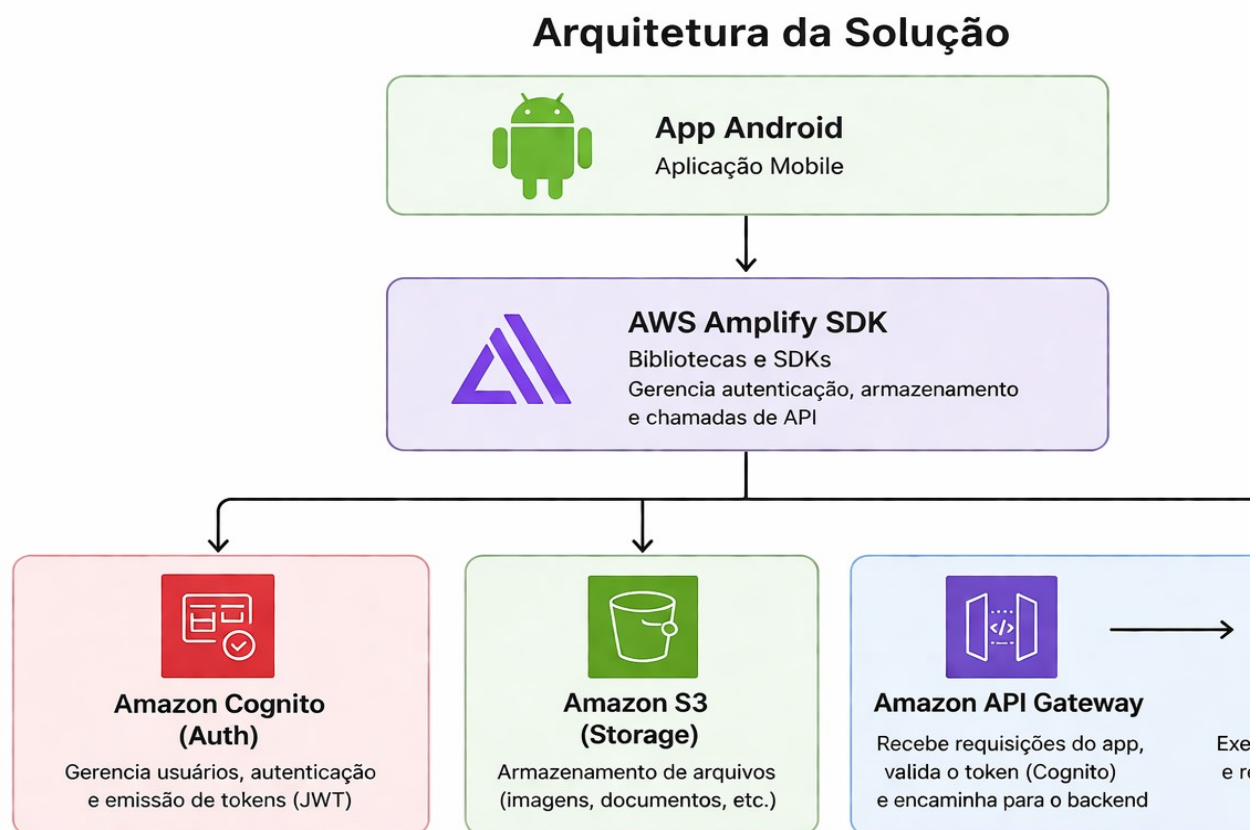


Figura 1 - Arquitetura de exemplo para a solução

A figura 1 mostra que o aplicativo *Android* utiliza o *AWS Amplify* como camada de integração com os serviços da *AWS*. O *Amplify Auth* envia as operações de cadastro e login para o *Amazon Cognito*. O *Amplify Storage* permite enviar arquivos para o *Amazon S3 (Storage)*. Já o *Amplify API* permite que o aplicativo realize chamadas para o *Backend*, que pode utilizar o *Amazon API Gateway* e funções *AWS Lambda* para executar regras de negócio.

Tabela 1 - Requisitos do estudo de caso

Requisito	Serviço Amplify
Permitir cadastro e login do usuário.	Auth / Cognito
Permitir envio de foto de perfil ou comprovante.	Storage / S3
Listar refeições disponíveis.	API / API Gateway
Criar pedido somente para usuário autenticado.	Cognito + API Gateway
Consultar status do pedido.	API / Lambda

O primeiro fluxo é o de autenticação. O usuário informa e-mail e senha no aplicativo. Em seguida, o app utiliza o *Amplify Auth* para enviar a solicitação ao *Amazon Cognito*. Caso os dados estejam corretos, o usuário passa a ter uma sessão autenticada. Com isso, o aplicativo pode liberar funcionalidades como realizar pedidos, consultar histórico e alterar dados da conta.

O segundo fluxo é o de armazenamento. Em alguns casos, o usuário poderia enviar uma foto de perfil, anexar um comprovante de pagamento ou outro arquivo relacionado ao pedido. Para isso, o aplicativo seleciona o arquivo local e utiliza o *Amplify Storage* para enviá-lo ao *Amazon S3*. Depois do envio, a identificação do arquivo pode ser associada ao usuário ou ao pedido.

O terceiro fluxo é o de comunicação com o *Backend*. Para consultar o cardápio, criar um pedido ou acompanhar o status da compra, o aplicativo realiza chamadas para uma *API*. Essa *API* pode ser acessada pelo *Amazon API Gateway* e encaminhar as requisições para funções *AWS Lambda*, responsáveis por executar regras de negócio, como calcular o valor do pedido, validar produtos disponíveis e atualizar o status da entrega.

Do ponto de vista técnico, o fluxo de funcionamento do sistema pode ser descrito da seguinte forma: inicialmente, o usuário realiza o login no aplicativo, e o *Amplify Auth* envia as credenciais ao *Amazon Cognito*. Após a autenticação, o serviço retorna um *token JWT*, que passa a ser utilizado nas requisições subsequentes.

Ao realizar uma ação como consultar o cardápio ou criar um pedido, o aplicativo envia uma requisição para a *API*, incluindo o *token* no cabeçalho da requisição. O *Amazon API Gateway* recebe essa requisição, valida o *token* por meio de um mecanismo de autorização e, caso esteja válido, encaminha a chamada para uma função *AWS Lambda*. Essa função executa a lógica de negócio e retorna a resposta ao aplicativo.

Esse fluxo evidencia a separação entre cliente e *Backend*, além de demonstrar o uso de autenticação *stateless*, característica comum em arquiteturas modernas baseadas em serviços.

4.1 RESULTADOS E VALIDAÇÃO

Para validar o funcionamento da arquitetura proposta, foram realizados testes simulando as principais operações do sistema, incluindo autenticação de usuários, envio de arquivos e consumo de *stateless* da *API*.

Durante os testes, foi possível observar que o processo de autenticação apresentou resposta consistente, com geração de *tokens* válidos pelo *Amazon Cognito*. No envio de arquivos, o *upload* para o *Amazon S3* ocorreu sem falhas, demonstrando a integração correta com o serviço de armazenamento.

As requisições realizadas por meio do *Amplify API* também apresentaram comportamento esperado, com respostas sendo retornadas pelo *Backend* após validação no *Amazon API Gateway*. O tempo de resposta das operações se manteve adequado para aplicações mobile, evidenciando a eficiência da arquitetura *serverless* adotada.

De forma geral, os testes indicam que a solução é funcional, escalável e adequada para aplicações que demandam integração com serviços em nuvem de forma simplificada.

5. ANÁLISE CRÍTICA E LIMITAÇÕES

Apesar das vantagens apresentadas, o uso do *AWS Amplify* também envolve limitações que devem ser consideradas. Uma das principais é a dependência do ecossistema da *AWS*, o que pode gerar um cenário de vendor lock-in, dificultando a migração para outras plataformas no futuro.

Além disso, a abstração fornecida pelo *Amplify*, embora facilite o desenvolvimento, pode reduzir o controle sobre a arquitetura e dificultar a implementação de soluções altamente personalizadas. Em sistemas mais complexos, pode ser necessário utilizar diretamente os serviços da *AWS* ou adotar um *Backend* próprio.

Quando comparado a outras soluções de *Backend* as a Service, como o *Firebase*, o *AWS Amplify* apresenta maior flexibilidade e integração com um ecossistema mais amplo de serviços. No entanto, o *Firebase* pode oferecer uma curva de aprendizado mais baixa e maior simplicidade em projetos menores.

Dessa forma, a escolha pelo *AWS Amplify* deve considerar fatores como complexidade do sistema, necessidade de escalabilidade, conhecimento da equipe e requisitos de infraestrutura. Em projetos de pequeno e médio porte, especialmente com equipes reduzidas, o *Amplify* se mostra uma solução eficiente. Já em cenários mais complexos, uma abordagem mais personalizada pode ser mais adequada.

6. CONSIDERAÇÕES FINAIS

No contexto do desenvolvimento mobile contemporâneo, a integração entre aplicações *Android* e serviços em nuvem deixou de ser um diferencial e passou a representar um requisito essencial. Funcionalidades como autenticação de usuários, armazenamento de arquivos e comunicação com o *Backend* estão presentes na maioria dos sistemas, e a forma como essas integrações são implementadas impacta diretamente na escalabilidade, manutenção e tempo de desenvolvimento das aplicações.

Nesse cenário, o *AWS Amplify* permite que o desenvolvimento seja mais orientado às regras de negócio e à experiência do usuário, reduzindo a complexidade operacional envolvida na construção de *Backends*.

Além disso, a utilização de arquiteturas *serverless* contribui para maior escalabilidade e redução de custos operacionais.

O estudo de caso proposto evidenciou que os serviços podem ser utilizados de forma integrada e complementar, atendendo diferentes demandas de uma aplicação real. A autenticação garante controle de acesso, o armazenamento viabiliza o gerenciamento de arquivos e a *API* possibilita a comunicação estruturada com o *Backend*, formando uma arquitetura coesa e alinhada aos princípios de soluções modernas baseadas em nuvem.

Entretanto, a adoção do *AWS Amplify* deve ser realizada de forma estratégica. Em projetos que demandam rapidez na entrega, equipes reduzidas ou menor complexidade arquitetural, a ferramenta se mostra altamente vantajosa. Por outro lado, em cenários que exigem maior controle sobre a infraestrutura ou customizações específicas, o domínio direto dos serviços da *AWS* torna-se indispensável.

Dessa forma, conclui-se que o *AWS Amplify* não substitui o conhecimento da arquitetura subjacente da *AWS*, mas atua como um facilitador que reduz a barreira de entrada e acelera o desenvolvimento, mantendo-se como uma solução relevante dentro do ecossistema de aplicações mobile modernas.

7. REFERÊNCIAS

AMAZON WEB SERVICES. *Quickstart: AWS Amplify for Android.* Disponível em: <https://docs.amplify.aws/android/start/quickstart/>. Acesso em: 30 abr. 2026.

AMAZON WEB SERVICES. *Set up Amplify Auth.* Disponível em: <https://docs.amplify.aws/android/build-a-Backend/auth/set-up-auth/>. Acesso em: 30 abr. 2026.

AMAZON WEB SERVICES. *Amazon Cognito user pools.* Disponível em: <https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools.html>. Acesso em: 30 abr. 2026.

AMAZON WEB SERVICES. *Authentication: Amplify Android.* Disponível em: <https://docs.amplify.aws/android/frontend/auth/>. Acesso em: 30 abr. 2026.

AMAZON WEB SERVICES. *Amazon S3 objects overview.* Disponível em: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingObjects.html>. Acesso em: 30 abr. 2026.

AMAZON WEB SERVICES. *Upload files: Amplify Storage Android.* Disponível em: <https://docs.amplify.aws/android/frontend/storage/upload-files/>. Acesso em: 30 abr. 2026.

AMAZON WEB SERVICES. *What is Amazon API Gateway?* Disponível em: <https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>. Acesso em: 30 abr. 2026.

AMAZON WEB SERVICES. *Set up Amplify REST API: Android.* Disponível em: <https://docs.amplify.aws/gen1/android/build-a-Backend/restapi/set-up-rest-api/>. Acesso em: 30 abr. 2026.

AMAZON WEB SERVICES. *What is AWS Lambda?* Disponível em: <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>. Acesso em: 30 abr. 2026.

APÊNDICE A - GLOSSÁRIO DE SIGLAS

AWS - *Amazon Web Services*. Conjunto de serviços de computação em nuvem oferecidos pela *Amazon*, permitindo o desenvolvimento, hospedagem e escalabilidade de aplicações.

AWS Amplify - Plataforma da *AWS* que fornece ferramentas e bibliotecas para facilitar a integração entre aplicações *mobile/web* e serviços de *Backend*.

SDK - *Software Development Kit*. Conjunto de ferramentas, bibliotecas e documentação que auxiliam no desenvolvimento de aplicações para uma determinada plataforma.

API - *Application Programming Interface*. Interface que permite a comunicação entre diferentes sistemas e aplicações.

REST - *Representational State Transfer*. Estilo arquitetural utilizado para comunicação entre sistemas por meio de requisições *HTTP*.

BaaS - *Backend as a Service*. Modelo de serviço que fornece funcionalidades de *Backend* prontas, permitindo foco no desenvolvimento do *frontend*.

JWT - *JSON Web Token*. *Token* utilizado para autenticação e autorização, contendo informações do usuário e suas permissões.

IAM - *Identity and Access Management*. Serviço da *AWS* responsável pelo gerenciamento de permissões e controle de acesso a recursos.

Auth - *Authentication*. Processo de verificação da identidade de um usuário.

Storage - Armazenamento. Refere-se ao armazenamento de dados e arquivos em sistemas locais ou na nuvem.

API Gateway - Serviço da *AWS* que permite criar, gerenciar e expor *APIs*, atuando como intermediário entre cliente e *Backend*.

AWS Lambda - Serviço *serverless* da *AWS* que executa código sob demanda sem necessidade de gerenciar servidores.

Amazon Cognito - Serviço da *AWS* responsável pela autenticação e gerenciamento de usuários, incluindo login e geração de *tokens*.

Amazon S3 - *Simple Storage Service*. Serviço de armazenamento de objetos na nuvem da *AWS*.

Amplify Auth - Módulo do *AWS Amplify* responsável pela autenticação de usuários, integrado ao *Amazon Cognito*.

Amplify Storage - Módulo do *AWS Amplify* responsável pelo armazenamento de arquivos utilizando o *Amazon S3*.

Amplify API - Módulo do *AWS Amplify* que permite a comunicação com *APIs* de *Backend*.

HTTP - *Hypertext Transfer Protocol*. Protocolo utilizado para comunicação entre cliente e servidor na web.

JSON - *JavaScript Object Notation*. Formato leve de troca de dados amplamente utilizado em *APIs*.

Serverless - Modelo de computação em nuvem no qual a infraestrutura é gerenciada automaticamente pelo provedor.

CRUD - *Create, Read, Update, Delete*. Conjunto de operações básicas para manipulação de dados.